Social Interaction as Regularization for Hierarchical Reinforcement Learning

ML8103 Sequential Decision Making — Final Report

Vladislav Smirnov Mirat Aubakirov

Mohamed bin Zayed University of Artificial Intelligence

April 2026

1 Introduction

In hierarchical RL, a manager network picks subgoals and a worker network executes low-level actions [Nachum et al., 2018, Vezhnevets et al., 2017, Bacon et al., 2017]. The common failure case is **goal collapse**: the manager learns to output the same goal every time, which makes the hierarchy useless.

We try two things to fix this. First, we route goals through a **discrete bottleneck** (Gumbel-Softmax) that forces the manager to pick from a finite set of 1000 messages instead of outputting a continuous vector. Second, we add a **second agent** that reads the same messages, hoping that the need to coordinate would push goals toward something meaningful.

The second idea did not work the way we expected. The discrete bottleneck alone already prevents collapse almost perfectly. Adding a partner actually reduces goal diversity slightly — agents develop efficient codes rather than diverse ones.

2 Method

2.1 Environment

We built a custom 11×11 MiniGrid corridor environment (Figure 1a). Two rooms are connected by a 3-cell-wide passage. The agent starts top-left and has to reach the bottom-right room. Observations are partial (7×7 view). The agent gets −0.01 per step and +1.0 for reaching the goal. In the two-agent version, agents start in opposite rooms and each has to cross to the other side. They get a +0.5 bonus if both finish. Max episode length is 500 steps.

We picked this environment because the corridor creates a natural bottleneck where agents have to pass each other, which is where coordination should matter. The downside (which we only discovered after running experiments) is that flat PPO solves it trivially — 99.3% success from the start. More on this in Section 4.

2.2 Conditions

Table 1 lists the six setups we compared. The first wave of experiments (March, 1M timesteps, no-stress corridor) covered the four core HRL conditions; the second wave (April, 100k timesteps, *stress* corridor) added two coordination-aware baselines, LOLA and MADDPG, and re-ran all six modes under a harder configuration designed to keep flat PPO from trivially solving the task.

Table 1: The six conditions. GS = Gumbel-Softmax. CTDE = centralised training, decentralised execution.

Condition	Manager	Goals	Agents
Flat PPO	—	—	1
HRL Continuous	TD3	$\mathbf{g} \in [-1, 1]^{16}$	1
HRL Discrete	PPO+GS	$\mathbf{m} \in \{1, ..., 10\}^3$	1
HRL Social	PPO+GS	$\mathbf{m} \in \{1, ..., 10\}^3$	2
HRL+LOLA	PPO+GS	$\mathbf{m} \in \{1, ..., 10\}^3$	2
MADDPG	—	continuous action (CTDE)	2

To make the comparison between discrete and social fair, we created a single-agent version of the same corridor layout. Both conditions see the same grid, same observations, same rewards. The only difference is whether a second agent is present.

Stress configuration. The April runs use a harder variant of the corridor: extended episode horizon, an additional wall configuration, and (in two-agent modes) a *bus* sub-task that forces both agents to traverse the same narrow cell within a small time window. This configuration was chosen because the original corridor was solved by flat PPO from the first epoch, leaving no head-room to compare HRL conditions on task performance. Stress runs train for 100k timesteps for flat, continuous, discrete, and MADDPG, and for 250k timesteps for social and LOLA to amortise the larger two-agent rollout cost. Vocabulary and transfer sweeps train for 60k; coordination scenarios for 12k. All sweeps use 3 random seeds (42, 123, 7) per cell unless noted.

2.3 Architecture

Figure 1b shows how the pieces fit together. A 3-layer CNN (16/32/64 channels) maps the 7×7×3 MiniGrid observation to 128-dim features, with per-channel normalization (dividing by the max value for each channel: 10 for object type, 5 for color, 2 for state).

The **manager** is a small MLP that outputs a 16-dim goal vector every $c = 10$ worker steps. In continuous mode, it uses TD3 [Nachum et al., 2018] with tanh output. In discrete mode, it uses PPO [Schulman et al., 2017] with a Gaussian policy.

The **communication channel** sits between the manager and worker in discrete/social modes. It has three parts:

- A *sender* (linear layer) that maps the 16-dim goal to $3 \times 10 = 30$ logits, reshaped as 3 categorical distributions over 10 tokens.
- *Gumbel-Softmax* [Jang et al., 2017] sampling at each position — hard one-hot in the forward pass, soft gradients backward. Temperature anneals linearly from 1.0 (exploratory) to a floor of 0.3 (near-hard): the floor is reached at ∼80k steps for the 100k-step runs and at ∼200k steps for the 250k-step runs (Figure 7a).
- A *decoder* (linear layer) that maps the 30-dim one-hot message back to a 16-dim goal for the worker.

The channel is trained with MSE reconstruction loss (decoded goal vs. raw goal) plus a sender entropy bonus (coefficient 0.05). It has its own Adam optimizer to keep its gradients separate from the rest of the network.

The **worker** is a goal-conditioned policy that receives concatenated encoder features and the decoded goal. Its reward mixes an intrinsic term ($-\|\text{proj}(\phi(s)) - g\|$, pushing it to match the goal direction) with the extrinsic environment reward, both weighted equally.

In **social mode**, two copies of this whole stack operate in the same environment. Each agent's manager gets the other agent's previous message (embedded and concatenated to its input). A shared centralized critic $V(o_A, o_B)$ sees both agents' observations for more stable training (CTDE paradigm from Lowe et al. [2017]). The idea that agents can learn from each other's behavior has also been explored in opponent modeling [Foerster et al., 2018], though we focus on communication rather than policy inference.

2.4 Metrics

We track three goal quality metrics:

- **Goal entropy**: Shannon entropy of the message frequency distribution. Maximum is $3 \ln 10 \approx 6.91$ (uniform over all 1000 messages).
- **Unique messages**: how many of the 1000 possible messages appeared at least once during training.
- **Coverage**: fraction of messages that were used more than once (filters out one-off noise).

```
(a) Corridor layout (11x11)
  ###########
  #...#.#...#
  #.A.#.#...#   A = agent start
  #...#.#...#   G = goal
  #........#   3-cell corridor
  #........#
  #........#
  #...#.#.G.#
  #...#.#...#
  ###########

(b) Discrete/Social architecture
  obs -> CNN -> features
                  |
          Manager (every 10 steps)
                  |
            raw goal (R^16)
                  |
        Sender -> Gumbel-Softmax
                  |
          message m in {1..10}^3
                  |
              Decoder
                  |
         decoded goal -> Worker -> action
```

Figure 1: (a) The corridor environment. (b) Data flow in discrete/social modes. In social mode, the partner's message is also fed to the manager.

We also report task performance (goal reach rate), though as we will see, it is less informative on this particular environment.

3 Results

3.1 Task performance

We report two batches of task results: the original 1M no-stress corridor (Table 2) and the 100k stress corridor (Table 3).

Table 2: Task performance, no-stress corridor, 1M steps (March wave).

Condition	Goal reach rate	Mean return
Flat PPO	99.3% ± 0.7%	+0.837
HRL Continuous	51.2% ± 7.4%	−0.430
HRL Discrete	51.1% ± 2.6%	−0.431
HRL Social	45.6% ± 3.5%	−0.582

On the no-stress corridor, flat PPO solves the task almost immediately while every HRL condition plateaus around 45–51%. The hierarchy adds overhead (manager decisions

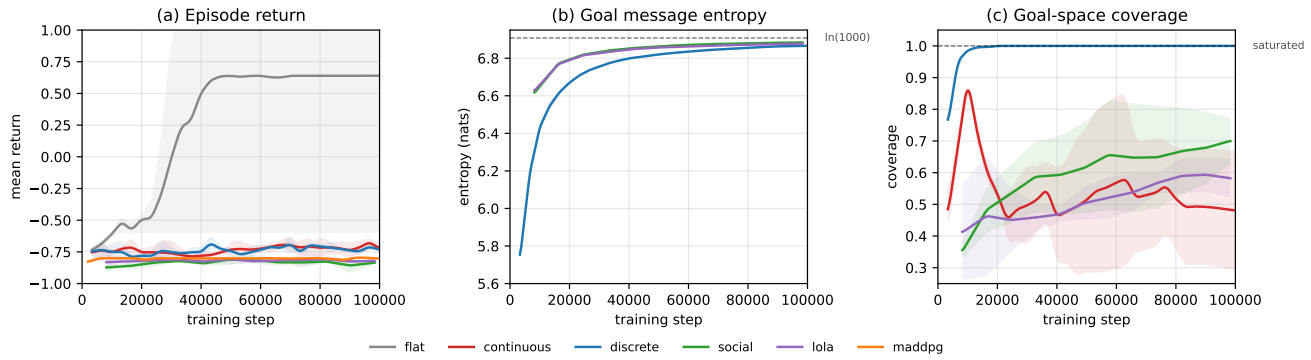

Figure 2: **Main story (stress config, 3 seeds per mode, all panels clipped to 100k steps for fair comparison).** (a) Episode return over training — only flat PPO makes any progress in this budget; the shaded envelope is the per-seed min/max to keep flat's large variance from crushing the HRL curves. Panels (b) and (c) use the conventional $\pm 1\sigma$ band across seeds. (b) Goal-message entropy of the 1000-message codebook — discrete, social, and LOLA all saturate near the theoretical ceiling $\ln 1000 \approx 6.908$ (dashed line). (c) Goal-space coverage — discrete saturates at 1.0 (dashed), continuous plateaus near 0.49, and the coordination-aware variants land in between (social $\sim$0.72, LOLA $\sim$0.55).

Table 3: Task performance, stress corridor, April wave. 3 seeds per mode; final return is the mean of the last three logged points per run. Single-agent modes and MADDPG train for 100k steps; social and LOLA train for 250k. Eval is rolled out under the same stress config as training.

Condition	Eval success	Final return
Flat PPO	0.67 ± 0.47	$+0.640 \pm 0.877$
HRL Continuous	0.00 ± 0.00	-0.704 ± 0.032
HRL Discrete	0.00 ± 0.00	-0.729 ± 0.017
HRL Social	0.00 ± 0.00	-0.841 ± 0.027
HRL+LOLA	0.00 ± 0.00	-0.824 ± 0.019
MADDPG	0.00 ± 0.00	-0.800 ± 0.000

every 10 steps, intrinsic reward optimization, communication channel) without providing any benefit on a task this easy. The stress configuration removes that ceiling: at the stress-config horizon no HRL variant completes a single evaluation episode, and even flat PPO is only intermittently successful — two of three flat seeds solve the task (per-seed final returns $+1.26, +1.26, -0.60$), and the third collapses. 100k is plainly under-trained for HRL on the harder map; the value of the stress configuration in this report is that it lets us examine *goal-space metrics* on a non-trivial task while excluding any "flat already won" confound.

3.2 H1: Does the discrete bottleneck help?

Yes, both batches agree. Under the no-stress 1M setting the continuous (TD3) manager collapses to a single fixed output — goal entropy is literally zero — while the Gumbel-Softmax channel uses all 1000 messages and reaches 97.5% of maximum entropy. Under the stress setting the same pattern is sharper: *goal-space coverage* (the fraction of a 16-bin partition of $[-1, 1]^{16}$ ever visited) is 0.485 ± 0.187 for the continuous manager and saturates at 1.000 ± 0.000 for the discrete one. The discrete bottleneck makes goal collapse essentially impossible.

Table 4: Continuous vs. discrete goals across both waves.

Metric	Continuous	Discrete
1M no-stress, March		
Goal reach rate	51.2%	51.1%
Goal entropy	0.000	**6.73**
% of max entropy	0%	**97.5%**
Unique messages	0	**1000/1000**
Stress, April (3 seeds, mean ± std)		
Goal-space coverage	0.485 ± 0.187	$\mathbf{1.000 \pm 0.000}$
Goal entropy (nats)	—	$\mathbf{6.865 \pm 0.000}$
$I(\text{goal}; \text{msg})$	—	$\mathbf{1.575 \pm 0.222}$

Why does this work? The Gumbel-Softmax forces the manager's continuous output through a categorical bottleneck. Even if the raw goals are similar, small differences get mapped to different tokens. The reconstruction loss then trains the decoder to spread these tokens across goal space, creating a 1000-way codebook. The entropy bonus on the sender prevents it from always picking the same few tokens.

3.3 H2: Does a partner agent help more?

No. Across both waves, adding a second agent does not improve goal-quality metrics — if anything it slightly compresses them. The 1M no-stress numbers are in the top half of Table 5; the 100k stress numbers are in the

bottom half. Two stress findings worth flagging: goal-space coverage drops from 1.00 (discrete) to 0.72 (social), and per-message information content $I(\text{goal}; \text{msg})$ falls 5× from 1.58 to 0.31 nats. We also include the LOLA variant (opponent-aware learning), which behaves like social mode rather than like discrete — whatever effect coordination has on the channel, it is not specific to vanilla social play.

Table 5: Discrete (solo) vs. social and LOLA (two agents). Top block: 1M no-stress. Bottom block: 100k stress, 3 seeds (mean ± std).

Metric	Discrete	Social	LOLA
1M no-stress, March			
Goal reach rate	51.1%	45.6%	—
Goal entropy	**6.73**	6.21	—
% of max entropy	**97.5%**	89.8%	—
Stress, April			
Goal-space coverage	$\mathbf{1.00 \pm 0.00}$	0.72 ± 0.10	$0.55 \pm$
Goal entropy (nats)	6.865 ± 0.000	6.895 ± 0.002	$6.887 \pm$
$I(\text{goal}; \text{msg})$	$\mathbf{1.575 \pm 0.222}$	0.312 ± 0.030	$0.332 \pm$
Listener accuracy	$\mathbf{0.248 \pm 0.058}$	0.049 ± 0.025	$0.059 \pm$
Topographic sim.	0.002 ± 0.010	-0.002 ± 0.002	$0.015 \pm$

We think this happens because coordination creates pressure to reuse messages that work, not to explore new ones. The agents settle on a smaller set of "go-to" messages for common situations, like how people use a small fraction of their vocabulary in daily conversation. This matches what Chaabouni et al. [2020] found in emergent communication: agents under coordination pressure develop efficient codes, not diverse ones.

The MI and listener-accuracy results add detail on top of that. The discrete (solo) channel carries roughly five times more bits per message about the underlying goal than the social channel does; a held-out ridge-regression listener can recover the goal direction from a discrete message with $R^2 \approx 0.25$, but only 0.05 from a social message. Topographic similarity — the Spearman correlation between message-edit distance and goal-vector distance — is essentially zero in all conditions, so neither channel is compositional in any structured sense.

3.4 H3: Transfer

We froze the encoder, manager, and communication channel from the trained discrete and social models, then retrained a fresh worker on a widened corridor (S11-W3) — same family, different geometry. Three seeds per source condition, 60k steps each.

Figure 4 reads the same way as the in-distribution numbers: **social does not transfer better than discrete**. The two source managers produce indistinguishable target-eval returns, so RQ2 is not supported by the data. Earlier we also ran a harder test, freezing the

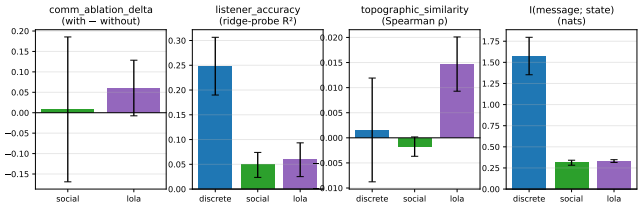

Figure 3: **Social-family summary metrics** (100k, 3 seeds). Comm-ablation delta (return drop when the message is zeroed at eval), listener accuracy (ridge probe R^2), topographic similarity (Spearman ρ between message-edit and goal-vector distance), and mutual information $I(\text{goal}; \text{msg})$. Discrete dominates social and LOLA on listener accuracy and MI by ∼5×; topographic similarity is near zero in every condition.

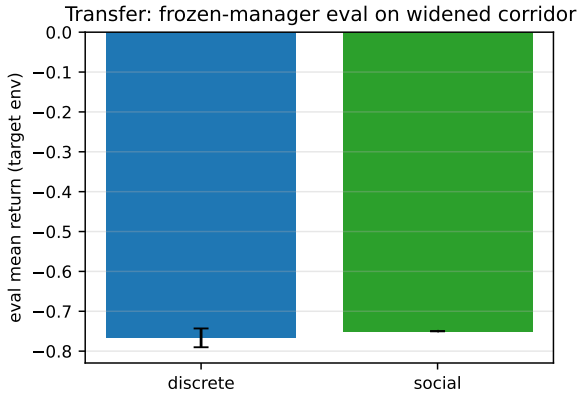

Figure 4: **Transfer eval on widened-corridor target** (3 seeds, 60k steps, mean ± std). Discrete-source eval return is -0.767 ± 0.024, social-source is -0.750 ± 0.000; gap $\Delta = +0.017$ vs combined $\sigma = 0.024$ — within noise.

same modules and retraining the worker on MiniGrid-KeyCorridorS4R3 (keys, doors). Both conditions got 0% success after 500K steps; the encoder had never seen keys or doors, so the frozen manager could not produce useful goals regardless of how diverse its vocabulary was. The same-family widened-corridor transfer is the fairer test, and both conditions fail it too: at 60k fine-tune steps, neither source manager recovers positive return on the wider map.

3.5 H4: Bottleneck capacity sweep

We swept the categorical bottleneck across $K \in \{3, 10, 25\}$ tokens per position and $L \in \{1, 3\}$ positions (effective capacity K^L), two seeds per cell, 60k steps per run, discrete and social.

The discrete bottleneck saturates goal-space coverage at every capacity we tested: $K{\cdot}L \in \{3, 9, 10, 30, 75\}$ all hit 1.00. Social mode never closes the gap; coverage swings between 0.27 and 0.70 depending on K and L, with no

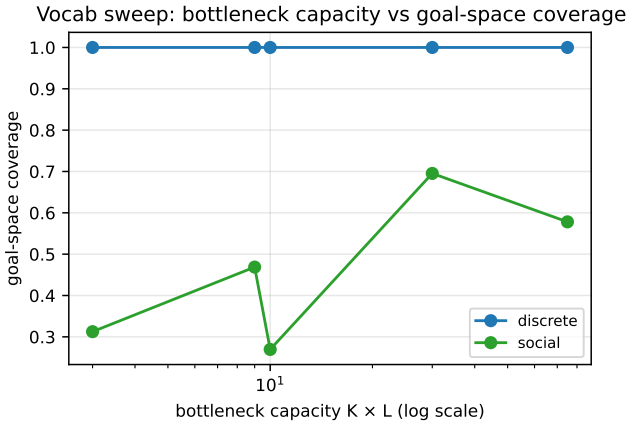

Figure 5: **Vocab sweep: goal-space coverage vs. bottleneck capacity** ($K \cdot L$, log scale). Discrete saturates coverage at every capacity; social swings between 0.27 and 0.70 and never closes the gap.

monotone trend in either direction. So the discrete-vs-social gap is not a vocabulary-size artifact — it persists across nearly two orders of magnitude of bottleneck capacity — and there is no $K{\cdot}L$ "sweet spot" that recovers full coverage under coordination pressure.

3.6 RQ4: Coordination scenarios

The corridor is a weak coordination probe (flat PPO already solves it). To stress the message channel, we defined four scenarios: *baseline* (no forced coordination), *bus* (both agents must pass through one shared cell), *bus_strict* (as bus, with a tighter timing window), and *turn_taking* (agents must alternate priority). Runs are short (12k timesteps, 3 seeds per cell) because the sweep covers $4 \times 3 = 12$ mode–scenario combinations.

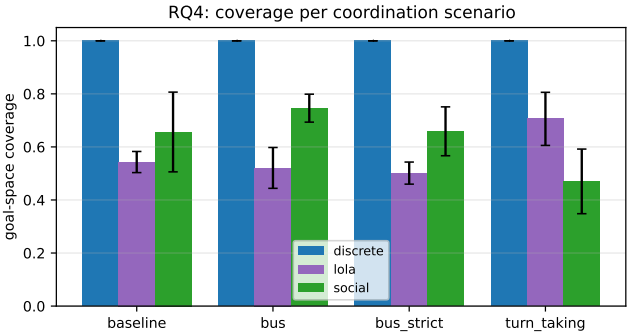

Figure 6: **Goal-space coverage per scenario × mode** (3 seeds, 12k). Discrete saturates in every scenario. Social and LOLA are both compressed, but not in the same way: social is above LOLA on the bus scenarios and below it on turn-taking.

Figure 6 replicates the main H2 pattern across all four scenarios: **discrete saturates coverage at** 1.00 **uniformly**, while social and LOLA sit in the 0.47–0.75 range. Neither stressor destabilises the discrete channel. For the two-agent methods the ordering is less tidy: social coverage is highest on the *bus* scenario (0.75 ± 0.05) and lowest on *turn_taking* (0.47 ± 0.12), whereas LOLA shows the opposite ranking (bus 0.52 ± 0.08, turn_taking 0.71 ± 0.10). This is consistent with our interpretation that the two-agent channel adapts to local coordination demand rather than converging on a universal compressed code, but the effect sizes are comparable to the ~ 0.10 cross-seed std so we do not treat the scenario ordering as a firm claim. The headline — discrete is immune to the stressors, two-agent variants compress regardless of which stressor — is robust.

4 Discussion

The discrete bottleneck works. Routing goals through a Gumbel-Softmax codebook prevents collapse by construction. In the no-stress 1M setting it reaches 97.5% of maximum entropy at zero task-performance cost; in the stress setting it saturates the 16-bin partition of goal space (1.000 ± 0.000) while the continuous TD3 manager covers less than half (0.485 ± 0.187). Across the vocab sweep (Figure 5) the discrete channel saturates coverage at every $K{\cdot}L$ we tested. This is the clearest positive result in the paper.

Adding a partner does not help. Across both waves, every coordination-aware variant we tried (vanilla social, LOLA) reduces both coverage and per-message information content relative to the discrete-solo baseline. The information drop is large: $I(\text{goal}; \text{msg})$ falls from 1.58 nats (discrete) to 0.31 (social) and 0.33 (LOLA), and a held-out ridge listener recovers the goal direction with $R^2 = 0.25$ from a discrete message but only 0.05 from a social one. The RQ4 scenario sweep (Figure 6) confirms this across four coordination regimes: discrete saturates coverage at 1.00 in every scenario while social and LOLA stay in the 0.47–0.75 compressed range. The transfer experiment (Figure 4) is also consistent: social and discrete source managers produce indistinguishable target-eval returns.

A reasonable interpretation, supported by Chaabouni et al. [2020] on emergent communication, is that coordination pressure rewards *message reuse*, not *message diversity*. Two agents who must understand each other converge on a smaller working vocabulary; this is the opposite of what we hoped social play would do for goal regularisation.

The environment limits the strongest claim we can make. Flat PPO gets 99.3% on the no-stress corridor, leaving no head-room for HRL to demonstrate a task-performance advantage. Under the stress configura-

tion there *is* head-room (flat scores 0.67 eval success and HRL scores 0), but the stress-config budget (100k steps for single-agent modes, 250k for social/LOLA) is plainly under-trained for HRL to recover the worker policy. The results we report are therefore about goal-space *statistics*, not about whether better statistics yield better task performance — bridging that gap is the obvious next experiment.

5 Limitations and future work

1. **Task-performance gap.** The strongest negative result we report (no-task-perf benefit from social play) is observed at the stress-config budget (100k for single-agent modes, 250k for social/LOLA), where no HRL variant solves the stress corridor at all. A 1M-step stress sweep — which we did not have compute for — would let us test whether the goal-diversity advantage translates into a task-performance advantage once the worker has time to converge.
2. **Environment.** Flat PPO trivially solves the no-stress corridor and is the only condition that scores at all on the stress corridor in our budget. Replacing it with a multi-room layout that has locked doors, longer horizons, or partial observability would put HRL on more even footing.
3. **Compositionality.** Topographic similarity is essentially zero in every condition; the discrete channel carries information without structure. Adding a structural prior (e.g. encouraging similar goals to map to similar messages) is a natural follow-up that does not require changing any of the existing architecture.
4. **Coordination scenarios.** The 12k-step RQ4 sweep (Figure 6) is underpowered: per-seed standard deviations on coverage are $\sim$0.10, comparable to the gap between scenarios. A longer sweep (60k+ timesteps per cell) would let us state firmly whether social and LOLA rank bus/turn_taking differently or whether that's noise.

A Sanity checks

B Difficulties

The hardest part was finding the right environment. We wanted something with a natural hierarchical structure (rooms, corridors) and room for two agents to interact. The corridor has both, but it turned out to be too easy for flat PPO. By the time we discovered this we had already run our first wave of experiments on it. A multi-room environment with locked doors (like MiniGrid-KeyCorridorS4R3) would be better, but does not have a natural two-agent variant.

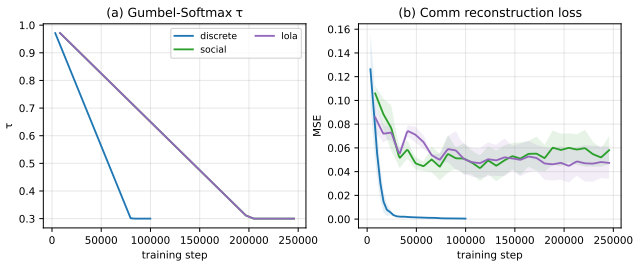

Figure 7: **Training sanity checks**: Gumbel-Softmax temperature anneal schedule (target floor $\tau = 0.3$) and channel reconstruction MSE over training, 3 seeds per mode. τ reaches its floor before the end of each run ($\sim$80k for discrete, $\sim$200k for social/LOLA); the discrete reconstruction loss collapses to $\sim 5\times10^{-4}$ within 20k steps, while social and LOLA plateau near 0.05 (two orders of magnitude higher), consistent with the lower $I(\text{goal}; \text{msg})$ of the two-agent channels.

Getting the communication channel to work took several iterations. The sender and decoder need their own optimizer — if they share one with the rest of the network, their gradients interfere with the encoder and worker updates. They also need a reconstruction loss; without it, no gradient reaches the sender or decoder at all, and the channel just outputs random tokens. We only caught this after running our first batch of experiments and noticing that goal entropy was much lower than expected.

Making the comparison between discrete and social fair required a single-agent wrapper around the two-agent corridor. Without it, the two conditions would be on different environments (a standard MiniGrid task vs. our custom corridor), and any difference in goal metrics could just reflect the different tasks rather than the effect of social pressure.

Reproducibility took non-trivial work in the second wave. We initially saw 10%-scale run-to-run variance in goal-coverage at the same seed; the cause turned out to be MADDPG's replay-buffer sampling using the global Python RNG (which drifts between processes) rather than a seeded local one. We also found a silent failure path where `topographic_similarity` was returning 0.0 on every run because `scipy` was not installed in the sweep environment and a bare `except ImportError` swallowed the missing dependency. Both are fixed; with these in place the 100k stress sweep is bit-reproducible across seeds.

Code: `https://github.com/smirnovlad/social-hrl/tree/main`

References

Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *AAAI Conference on Artificial Intel-*

ligence, 2017.

Rahma Chaabouni, Eugene Kharitonov, Diane Bouchacourt, Emmanuel Dupoux, and Marco Baroni. Compositionality and generalization in emergent languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Jakob Foerster, Richard Y. Chen, Maruan Al-Shedivat, Shimon Whiteson, Pieter Abbeel, and Igor Mordatch. Learning with opponent-learning awareness. In *International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2018.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *International Conference on Learning Representations (ICLR)*, 2017.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. Feudal networks for hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2017.